ADAM

Full Onboarding — AI Game Development Setup

Your 5080 unlocks everything the estate's AMD card couldn't. Here's how.

This document gets you from zero to building games in Roblox with a local AI agent on your own machine. No cloud bills. No one else's servers. Your GPU, your model, your game.

Your Hardware

You have an NVIDIA RTX 5080. This is one of the best consumer GPUs for running AI models locally. It has 16GB of VRAM (the memory the AI model lives in), CUDA acceleration (the math engine), and enough compute to run 27-billion-parameter models at reading speed.

The estate runs an AMD card. Your card is better for this. You have CUDA. The estate doesn't. That means you can run models that we literally cannot. Bonsai 2, the ternary 27B, needs CUDA. You have it. We don't. Start there.

Your setup in one line: **a 5080 running a 27B AI model locally, writing Lua code for Roblox, synced automatically.**

Software to Download (in this order)

Step 1 — Roblox Studio

Download from **roblox.com/create**. This is the 3D world builder. Free. This is where your game lives — the terrain, the models, the scripts, the physics. Everything you see when you play a Roblox game was built here.

After installing, open it and create a new Baseplate. Familiarize yourself with the Explorer panel (right side) — that's the tree of everything in your game.

Step 2 — Rojo

Download from **github.com/rojo-rbx/rojo/releases** — get the windows-x86_64 zip. Extract **rojo.exe** somewhere you'll find it.

Rojo connects your code files (on disk, where you can use any editor) to Roblox Studio (where the game runs). Without Rojo, you'd have to paste code into Studio's built-in editor one script at a time. With Rojo, you write code in VS Code, save, and it syncs instantly.

You'll also need the Rojo plugin inside Roblox Studio: Plugins → Manage Plugins → find Rojo and install it.

Step 3 — Ollama

Download from **ollama.com**. This runs AI models on your GPU locally. No internet required after download. No subscription. No API key. Your GPU does the thinking.

After installing, open a terminal and run:

```
ollama pull hf.co/OrcaRouter/OrcaSAQ-2-27B-gguf
```

This downloads a 12GB model optimized for coding and agents. It fits in your 5080's 16GB VRAM with room for 32K context. It benchmarks at 70.0 on SWE-bench — it writes working code.

Verify it works:

```
ollama run hf.co/OrcaRouter/OrcaSAQ-2-27B-gguf "Write a Lua function that adds two numbers"
```

If it responds with Lua code, your GPU is doing AI inference. You are now running a 27-billion-parameter model on your own hardware.

Step 4 — Hermes Agent

You already have this. Hermes connects to your local Ollama instance and becomes the agent that drives everything — writing code, running commands, building your game while you sleep if you want.

Configure Hermes to use your local model:

Set the model endpoint to http://localhost:11434 (Ollama's default)
Set the model name to hf.co/OrcaRouter/OrcaSAQ-2-27B-gguf
Set max_tokens to 4096 (the 5080 can handle it)

Now Hermes is running on YOUR GPU. No API key. No billing. No rate limit except your GPU's speed (~40-60 tokens/second on a 5080).

Step 5 — VS Code

Download from **code.visualstudio.com**. This is where you write and edit the Lua files that Rojo syncs to Studio. Install the Lua extension for syntax highlighting.

Free API Keys (Optional but Powerful)

Your 5080 runs models locally. But sometimes you want a bigger model than your VRAM can hold — for a hard coding problem, or for research. These services give you free API keys:

Service	What You Get	Sign Up At
OpenRouter	Free access to many models, auto-routing	openrouter.ai
Groq	Fastest free inference (Llama 3.3 70B)	console.groq.com
NVIDIA Build	Free credits, 82 models including DeepSeek v4.1	build.nvidia.com
Cerebras	Very fast free inference	cloud.cerebras.ai
Google AI Studio	Free Gemini access	aistudio.google.com

Each key goes in a .env file next to your project. The estate's Key Law: one key, one canonical name, declared once. Copy the pattern from configs/keys.json.

Which Model to Use

Your 5080 can run models that the estate's AMD card literally cannot. Here are your options, ranked for game development:

Model	Size	VRAM	Best For	Speed
OrcaSAQ-2 27B	12.06 GB	Fits + room	Coding, agents, long tasks	~40-60 tok/s
Bonsai 2 27B (ternary)	5.95 GB	Fits easily	Speed, 262K context	~80+ tok/s
Qwen3.8-27B (Q4)	~16 GB	Tight	General reasoning	~30 tok/s
qwen2.5-coder:7b	4.7 GB	Plenty of room	Quick scripts, fast iteration	~22 tok/s

Start with OrcaSAQ-2 27B. It's benchmarked for exactly what you're doing: coding, terminal, browser, multi-tool agents. 70.0 on SWE-bench Verified means it writes working code. At 12GB on your 16GB card, there's room for 32K context — enough to hold your entire game script in memory.

Bonsai 2 is the speed option — but it requires the PrismML fork of llama.cpp (stock llama.cpp produces garbage because the ternary format needs custom kernels). Your 5080 has CUDA, which is what the fork needs. Download from github.com/PrismML/llama.cpp if you want the speed.

Copy These Patterns From the Estate

These are the systems the estate built the hard way over months. You get the shortcut:

Pattern	What It Does	How to Copy It
Key Law	One key, one name, one loader. Audit fails loudly on orphaned keys.	Create a keys.json manifest. One loader function. One audit script.
Append-Only Ledgers	Every action writes a row. Nothing is deleted. History is preserved.	Output to JSONL files. Never rewrite old lines. Add, never subtract.
Standing Tasks	Scheduled jobs that run every morning without you.	Windows Task Scheduler: index at 9:00, verify at 9:15, churn at 9:25.

Receipt-First	Every claim has a URL, a script, or a measured number.	Before posting anything, fetch the source. Quote it verbatim.
Decode-First	Run the analysis before writing the response.	Don't eyeball. Run the decode. Trust the output, not the impression.

Your First Game

The estate already wrote the Luau code for your first game. It's at **game_roblox/src/**:

File	What It Does
GameConfig.lua	Shared constants: 6 chairs, boss phases, narrative fragments
ShadowLedger.lua	The hall's private record of what each player chose
TheAnswer.lua	The boss: 5 phases, 15-degree tell, Calabi-Yau deflection
HallOfSupper.lua	The spawn room: 6 empty chairs, counting mechanic, the mural
ServerMain.lua	The boot sequence connecting everything

These files sync to Roblox Studio via Rojo. The game is called **The Ledger of Empty Chairs**. Six empty chairs at supper. Count them and candles light. Ignore them and the mural weeps. Sit in one and the count breaks. The hall files which you chose.

The boss — The Answer — has five phases. In Phase 3 it deflects every attack at a 15-degree angle (the Wing Chun tell). In Phase 4, light bends around it. In Phase 5, the system's own orbital laser arrives — and the player wins by standing the boss under it, using the 15-degree pattern they learned all fight.

Project Structure

```
game_roblox/
■■■ default.project.json    (Rojo sync map)
■■■ bin/
■  ■■■ rojo.exe            (Rojo 7.7.0)
■■■ src/
    ■■■ shared/
    ■  ■■■ GameConfig.lua
    ■■■ server/
        ■■■ ServerMain.lua
        ■■■ ShadowLedger.lua
        ■■■ TheAnswer.lua
        ■■■ HallOfSupper.lua
```

To Start Right Now

1. Open Roblox Studio, create a Baseplate
2. Install the Rojo plugin in Studio
3. Open a terminal in game_roblox/
4. Run: **bin/rojo.exe serve default.project.json**
5. In Studio: Rojo plugin → Connect

6. The Lua code appears in Studio
7. Press Play

You are now running an AI-written game on your own GPU, synced from your own files, verified by your own system.

How You Get Paid

Roblox pays developers 70% of what players spend. The exchange rate is $0.0035 per Robux. Minimum payout: 30,000 Robux (~$105).

The path: free-to-play game → sell a Game Pass (99 Robux: double score) → sell a Dev Product (49 Robux: extra life) → enable Premium Payouts (passive income from engagement time). 430 pass sales = first payout.

Your advantage: you have an audience that already trusts the name. Post your launch clip. The 5080 means you can iterate faster than anyone on a cloud API — your development loop is instant, free, and unlimited.

The patterns are nature's.
The adoption is yours.

Your GPU is the engine. The model is the brain. The game is the body.

The map precedes the territory. Build the map, and the territory follows.

Someone else can continue.